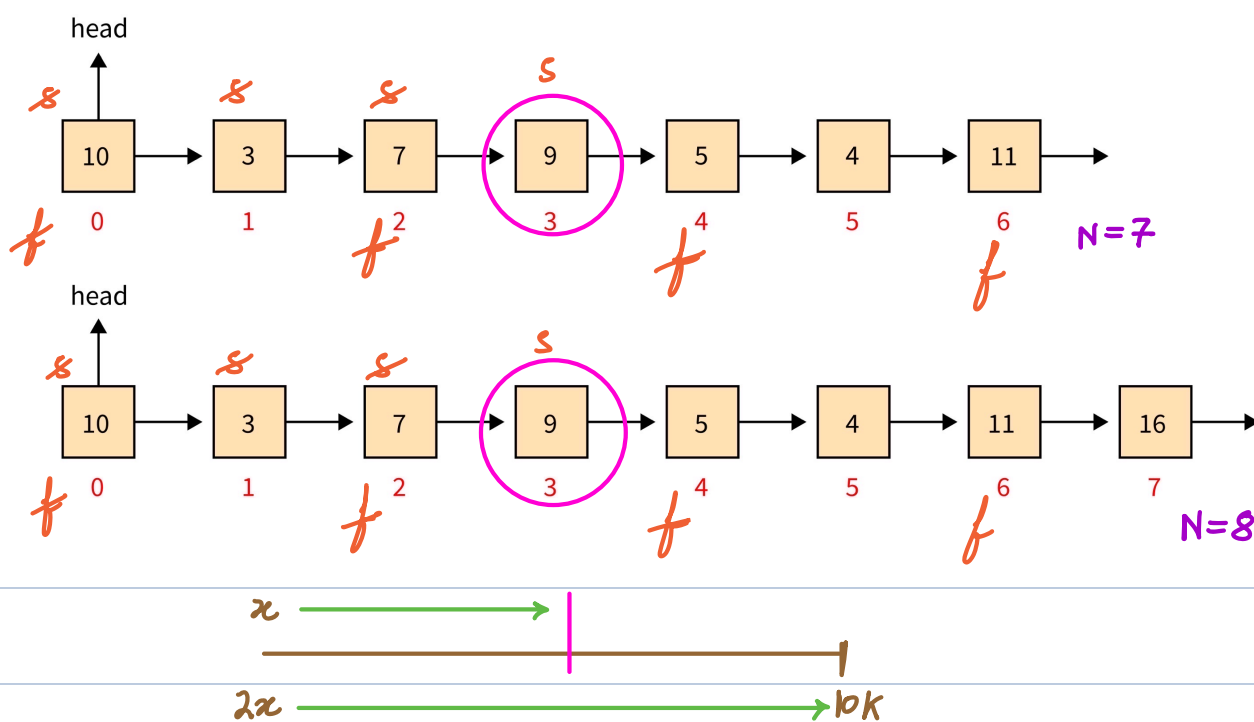


Agenda:

1. Merge Two Sorted List
2. Merge Sort on Linked List
3. Check if given linked list has cycle
4. Find the start of the cycle

Find Middle Element



Sol1 → 1) Travel the LL to find its length (ie N).
2) Travel $(N-1)/2$ nodes to reach middle.

temp = Head N = 0

while (temp != null) {

 N++
 temp = temp.next
}

start end
|—————→|

temp = Head

for i → 1 to (N-1)/2 {

 temp = temp.next
}

start mid
|————→

TC = O(N)

SC = O(1)

return temp

Sol 2 →

s = Head f = Head

while (f.next != null && f.next.next != null) {

 s = s.next
 f = f.next.next
}

return s

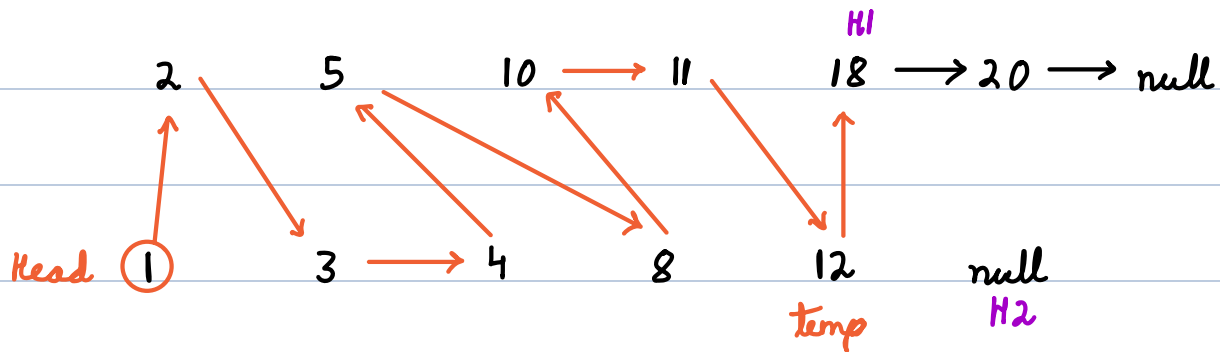
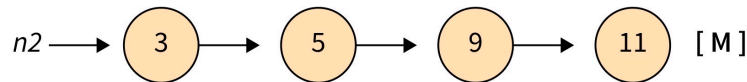
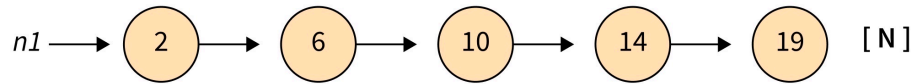
TC = O(N)

SC = O(1)

Merge Two Sorted List

Q1: Given **two sorted** Linked Lists, merge them into a **single sorted linked list**.

(do not create new linked list)



if ($H1 == \text{null}$) return $H2$

if ($H2 == \text{null}$) return $H1$

if ($H1.\text{data} \leq H2.\text{data}$) {

$\text{Head} = H1$

$H1 = H1.\text{next}$

} else {

$\text{Head} = H2$

$H2 = H2.\text{next}$

}

```

while (H1 != null && H2 != null) {
    if (H1.data <= H2.data) {
        temp.next = H1
        H1 = H1.next
    } else {
        temp.next = H2
        H2 = H2.next
    }
    temp = temp.next
}

```

```

if (H1 == null) temp.next = H2
else temp.next = H1
return Head

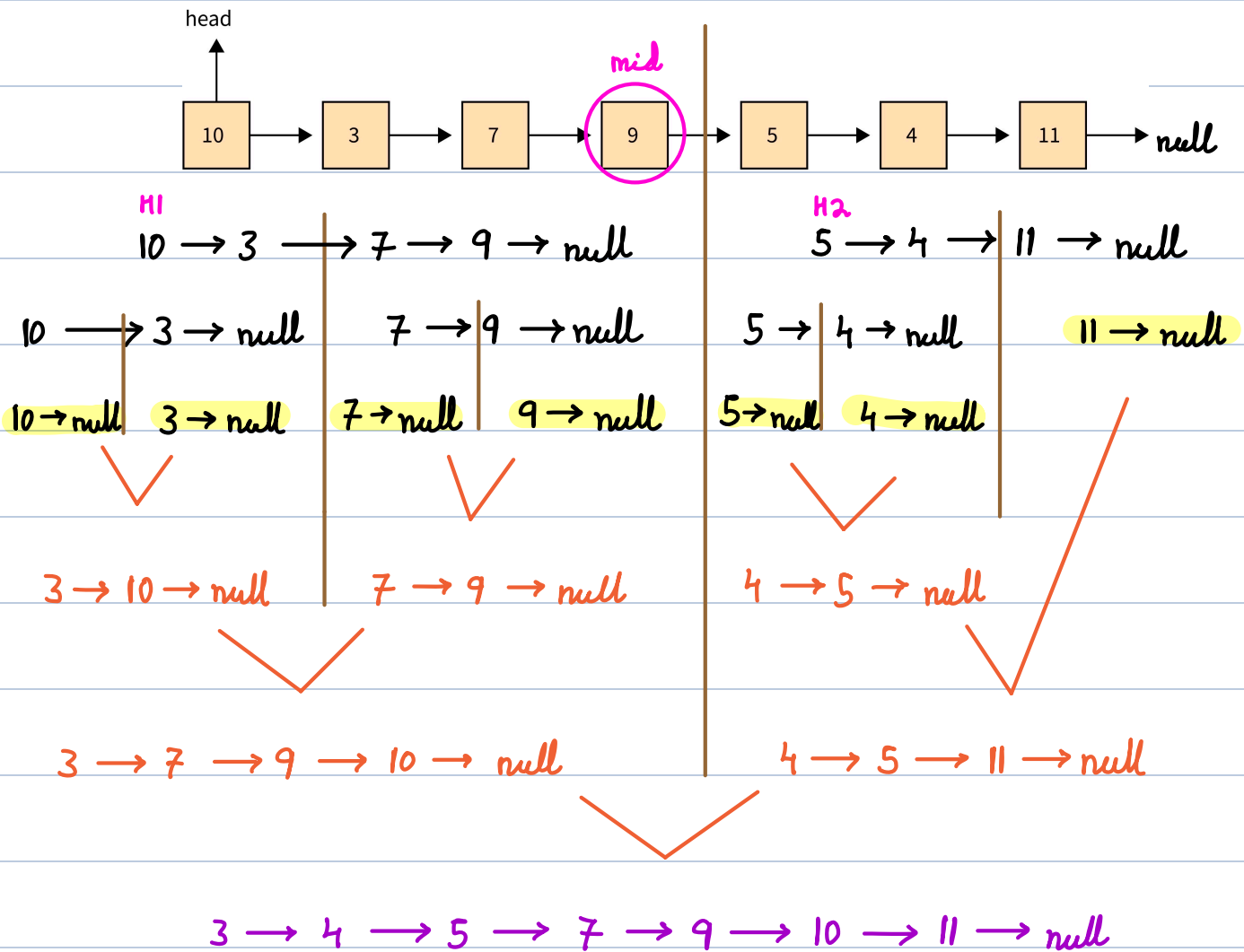
```

$$TC = \underline{O(N+M)}$$

$$SC = \underline{O(1)}$$

Merge Sort on Linked List

Q2: Sort the given linked list using merge sort.



Node sort (Head) {

if (Head == null || Head.next == null)

return Head

[mid = getMiddle (Head) → TC = $O(N)$

H1 = Head H2 = mid.next

[mid.next = null

H1 = sort (H1)

H2 = sort (H2)

return merge (H1, H2) → TC = $O(N)$

}

$$TC = \underline{O(N \log(N))}$$

$$SC = \underline{O(\log(N))}$$

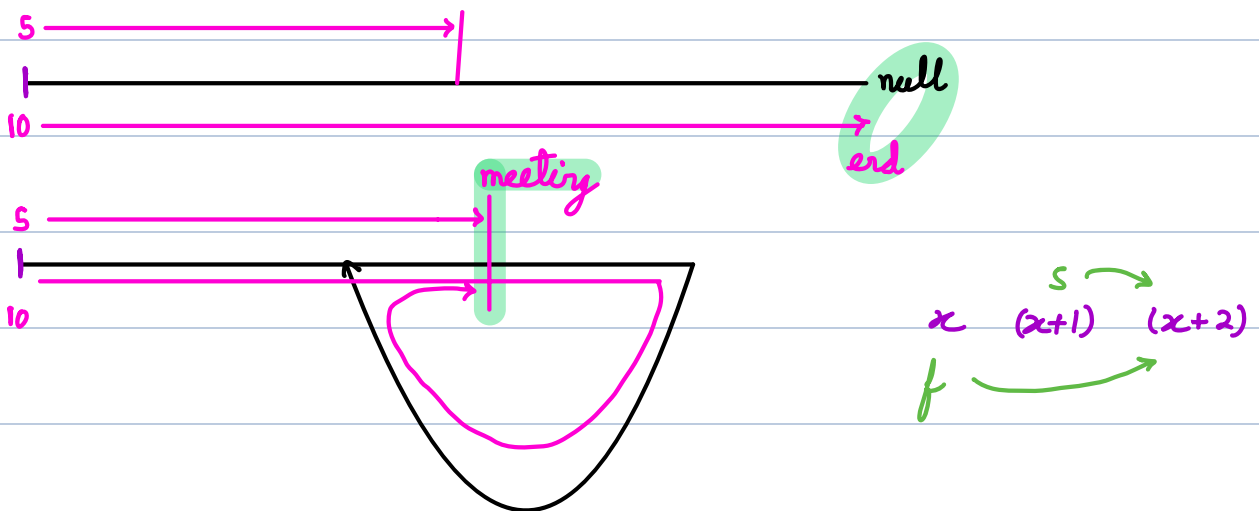
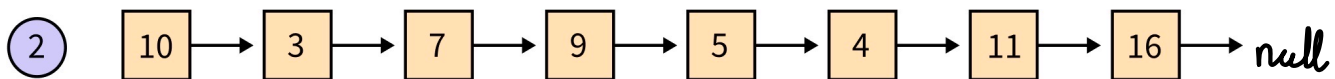
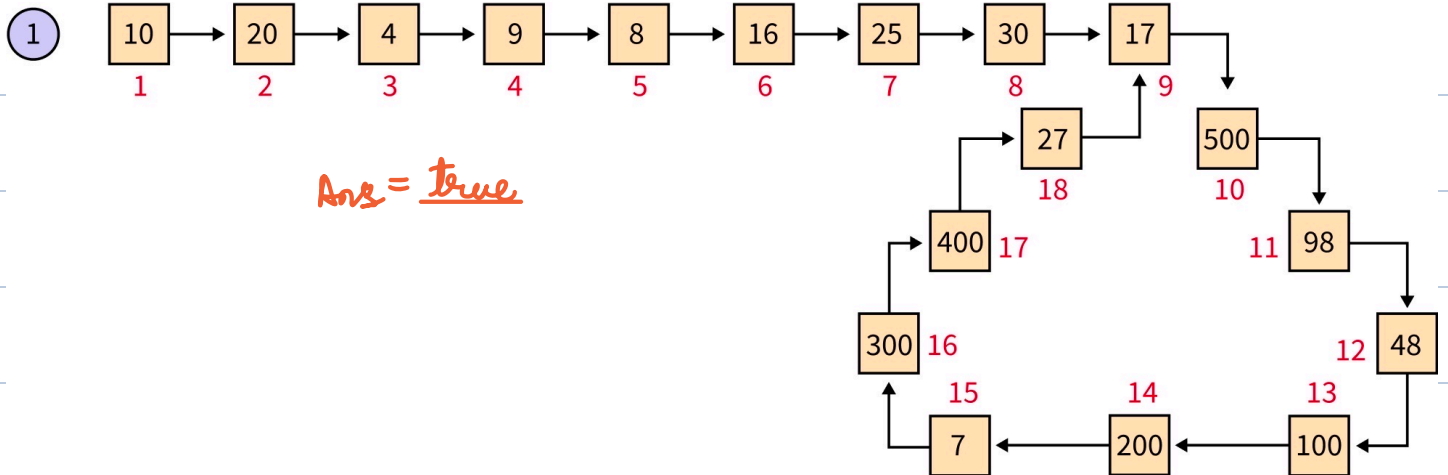
Check if given linked list has cycle

Q3: Given a linked list, determine whether the list contains a cycle.

✓ A cycle exists when the last node's next pointer leads back to a previous node, forming a loop.

✗ If no such loop exists, the list ends with NULL.

$$SC = O(1)$$



s = Head f = Head

while (fast != null && f.next != null) {

|

s = s.next

f = f.next.next

if (s == f) return true

}

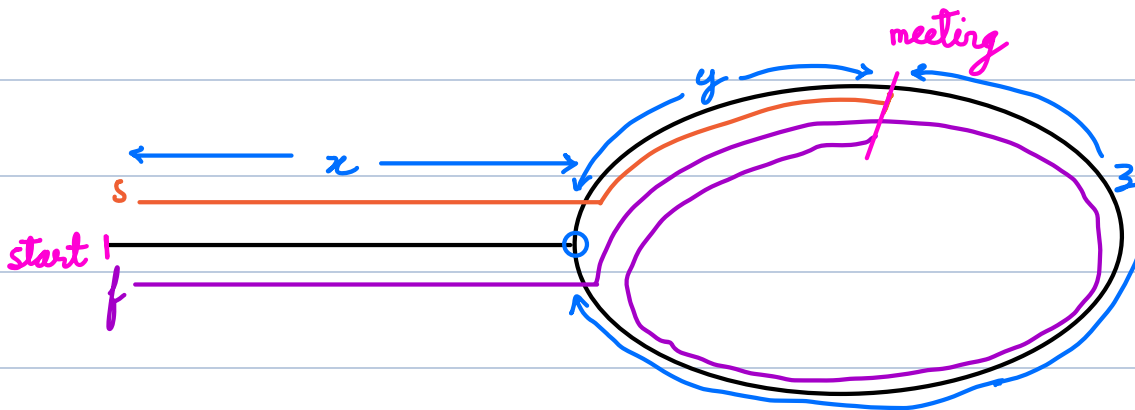
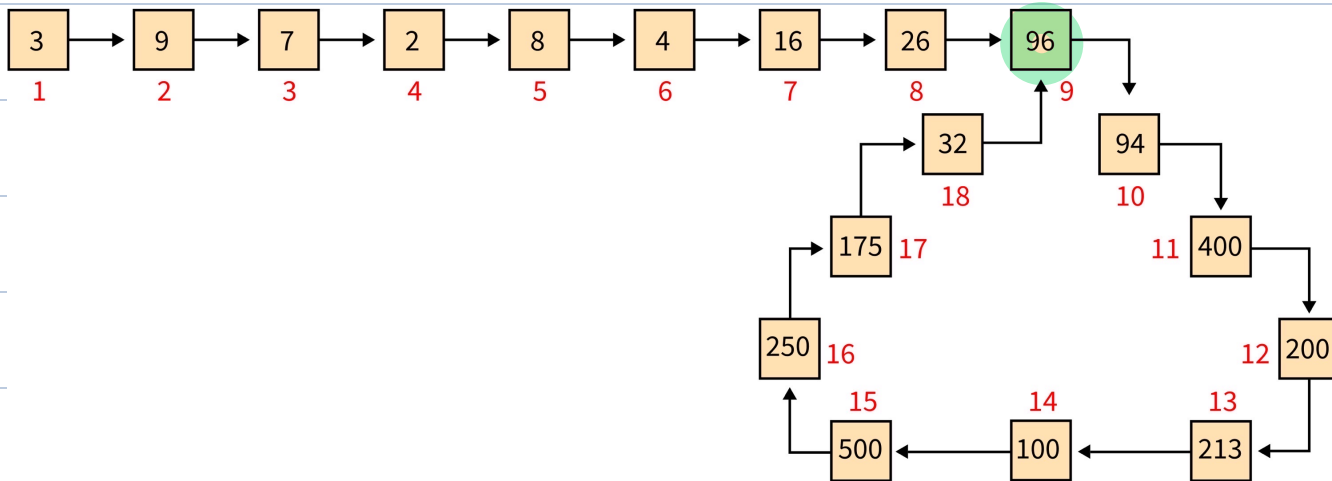
return false

TC = O(N)

SC = O(1)

Find the start of the cycle

Q4: Given the head of a singly linked list that contains a cycle (loop), find the node where the **cycle begins** (the first node that is part of the loop).



Distance travelled by

$$\text{slow} = x + y$$

$$\text{fast} = x + y + z + y$$

Distance travelled by slow * 2

= Distance travelled by fast

$$(x + y) * 2 = x + y + z + y$$

$$\cancel{x + y} + \cancel{x + y} = \cancel{x + y} + z + \cancel{y}$$

$$\Rightarrow x = z$$

s = Head f = Head

while (fast != null && f.next != null) {

 s = s.next

 f = f.next.next

 if (s == f) break

}

u = Head

v = s

while (u != v) {

 u = u.next

 v = v.next

}

return u

TC = O(N)

SC = O(1)

Predict output of following code:

```
public class Test {  
  
    static boolean hasCycle(Node head) {  
        Node slow = head;  
        Node fast = head;  
  
        while (fast != null && fast.next != null) {  
            slow = slow.next;  
            fast = fast.next.next;  
  
            if (slow == fast)  
                return true;  
        }  
        return false;  
    }  
  
    public static void main(String[] args) {  
  
        LinkedList: 1->2->3->4  
        Head Node = a = Node 1  
  
        System.out.println(hasCycle(a)); → false  
    }  
}
```